Lab 2 - Perl and Regular Expressions

CMSC 6940, Fall 2010

Michelle Shaw

To cover...

- regular expressions with bash:
  - example 1 - regexps with linux commads find and grep
  - example 2 - regexps in a script with grep
  - example 3 - modify complex regex
- starting with perl:
  - example 1 - the hello world script
  - example 2 - more complex...
  - example 3 - associative arrays
- perl and regex:
  - example 1 - script to strip filename extensions

RE Example 1 - REs with grep and find

- let's start with some simple command line searches using different REs:
  - grep -i "cpmd" /usr/local/*
  - ls -lrth /usr/lib64 | grep -i "stdc++"
  - find /usr/lib64 -name "*lapa*so*"
  - find $HOME -maxdepth 1 -name ".*rc"
  - grep -Ei '(strict|warnings)' *.pl | wc -l
- some more complex REs:
  - grep -Ei "^[aeiou].*" *.pl
  - grep -i "^u+" *.pl
  - grep -Ei "^u+" *.pl (why does this one work?)
  - grep -i "$#" *.pl
  - grep -i '$#' *.pl (why does this one work?)
  - find $HOME -maxdepth 2 | egrep -i "\.+s"

RE Example 2 - grep and file contents

- search for occurrences of strings in some shell scripts
- shows a simple example of a regular expression in a shell script and also the use of regexps with grep
- instructions - write a shell script that:
  - takes in the command to run (grep in this case), an option to that command and the files to search through
  - checks to make sure the user puts options when using it and reports proper usage if they don't
  - checks to make sure there aren't too many options and reports the error to the user if there are
  - takes in the options to the script to reconstruct the command to run
  - runs the command and reports the exit status

RE Example 2, cont'd...

- NOTE: to provide a list of files, you will need to put them in double quotes or the shell will not process it properly
- once the script is written, play with the command and the regexp
- script will allow you to see the effects of different regexps
- shows one way to make sure that the right number of options are specified when using the script
- problem - if you don't want options to grep, you need to fix the script.
  - can you think of a way to write the script so it has a variable number of options?

RE Example 3 - modify complex RE

- take random string code and modify the REs to see the effect (you need to add this to a shell script file):

```
#!/usr/bin/bash
scratch="`cat /dev/urandom | \ uuencode -m /dev/stdout | \ head -2 | \ sed -n -e '2s/^\(.....\).*$/\1/p' | \ sed -e 's/\//_/g' -e 's/\\\/-/g' -e 's/&/=/g' | \ sed -e "s/\'/X/g" -e "s/\\\`/Y/g" -e "s/\\\"/Z/g"`" echo "string is: $scratch"
```

- change the number of "." in the first sed expression to print out more or less characters
- change the second regexp sets (start with -e opt to sed) to allow &
- change the third set to exclude "B"
- try a few of your own

Perl Example 1 - a simple script

- the infamous "hello world" program:
  - write a perl script to print "hello world"
  - run the script
  - edit the script to add two numbers defined inside the script and print the result
  - edit the script to read the two numbers as arguments to the script
  - edit the script to take an option to define whether or not to add or subtract the two numbers and to do the proper action when it finds the option
  - edit your script to add error checking:
    - if the options and arguments don't exist, spit back usage
    - if any of the options or arguments are wrong, print an error message that indicates the problem

Perl Example 2 - a more complex script

- now, look at file I/O, arrays, chomp, split, and the while loop
- write a script to:
  - take in a file as an argument
  - open the file or die with an error if it cannot be opened
  - use a while loop to read in the file, line by line, into an array and parse into string segments
  - print string fragments
  - close the file
- modify the script to indicate usage if the file is not specified as an argument to the script

Perl Example 3 - Associative Arrays

- read in cpi.txt, store acronym as hash key and string parts as the array
- reconstruct sentences
- hmm, this one is tricky so let's go through it together...
  - the usual preamble...

```
#!/usr/bin/env perl
use strict;
use warnings;
```

  - file open and close

```
open IFILE, "<", "cpi.txt" or die "ERROR: cannot open file cpi.txt
      for reading! \n";
...
close IFILE;
```

  - the while loop...

```
while ($line = <IFILE>) { ... }
```

Perl Example 3, cont'd...

- have 2 delimiters (: and a space), so split twice:

```
chomp($line);
@getkeys = split(":", $line);
$keyfield = $getkeys[0];
$arrfield = $getkeys[1];
@ent = split(" ", $arrfield);
```

- now the fun - assign the word "array" to a hash value

```
$cpi_ent{"$keyfield"} = [@ent];
```

- ... and the finale - reconstruct the strings

```
for my $key (keys %cpi_ent) {
    print "$key: ";
    for my $i ( 0 .. $#{ $cpi_ent{$key}}) {
        $val = $cpi_ent{$key}[$i];
        print "$cpi_ent{$key}[$i] ";
    }
    print "\n";
}
```

Perl Example 3, cont'd...

- ok, an important new concept, but one we won't cover - referencing in Perl
- without the [], the "array" would not be preserved and the hash value would be a scalar:
  - strict in perl does not allow you to assign arrays to scalars (exactly the way it should be!)
  - so, we need to preserve the "array" characteristic in the hash
  - ***you don't need to know referencing details for the assignments and exam***, but I include the script because it still has important concepts for you to know (such as assigning and accessing hash elements that are arrays)
    - it is also a more practical example than simply assigning values inside the script

Perl Example 3 - simpler example

```
#!/usr/bin/env perl

use strict;
use warnings;

my %jobdirs = (
    cm => [ "cpmd_jobs", "vasp_test" ],
    bioch => [ "gromacs_jobs", "namd_jobs" ],
    chem => [ "gauss_test", "nwchem_jobs", "gamess_jobs"]);
my $jobdir;

for my $jobtype (keys %jobdirs) {
    print "job type is $jobtype \n";
    for my $i ( 0 .. $#{ $jobdirs{$jobtype}}) {
        $jobdir = '/home/dshaw/'.$jobdirs{$jobtype}[$i];
        print "$jobdir has: \n";
        system("ls -lrth $jobdir");
    }
}
```

Perl and REs - Example 1

- write a script which takes the name of a file and strips off the extension
  - assume that there is no path in the filename
  - try the RE yourself, but if you cannot sort it out, you can look at the answer on courtenay in:
    /globalscratch/dshaw/CMSC6940/scripts/re_examples/re6.pl
  - print out the resulting filename without its extension